Rank 12 — Knowledge Relationship Discovery
Capability: Knowledge Relationship Discovery
Purpose: Reveal Hidden Connections
KMS Area: Tier 2 — Organizational Intelligence

Overview

The purpose of Knowledge Relationship Discovery is to help the KMS identify meaningful connections between organizational knowledge that users may not discover through normal search.

The KMS already does a good job of answering questions such as *"Find documents related to authentication"* through semantic and hybrid retrieval. However, finding similar documents and understanding how organizational knowledge is connected are two different problems.

For example, two documents may never use the same wording but may still be connected because they reference the same project, discuss the same entities, support the same decision, or describe problems that resulted in the same lesson learned.

The PRD describes this capability as **"Reveal hidden connections."**

The goal of Rank 12 should therefore be to create a reusable relationship intelligence layer on top of the knowledge already captured by the KMS.

What the Current KMS Already Gives Us

One important finding from reviewing the current implementation is that this feature does **not** need to start from zero.

The KMS already contains several forms of relationship information.

At a high level, the existing system already knows about:

- documents and document embeddings;
- topics and document-topic associations;
- projects and document-project associations;
- entities extracted from documents;
- decisions and other structured knowledge;
- experts and topic expertise;

- lessons learned;
- document relationships;
- organizational and departmental context.

The existing DocumentRelationship structure is especially important because it already provides a natural place to represent relationships between documents, including relationship type, weight/confidence, and supporting evidence.

This means Rank 12 should primarily improve **how relationships are discovered and scored**, rather than introducing another disconnected relationship model.

Proposed Approach

I recommend implementing relationship discovery as a **multi-signal pipeline**.

A common mistake would be to treat embedding similarity as the relationship itself.

For example:

Document A is 89% semantically similar to Document B → therefore they are related.

That can be useful, but it is not sufficient for an enterprise knowledge system.

Instead, embeddings should identify **candidate relationships**, while additional organizational signals determine whether a meaningful connection actually exists.

The high-level process would be:

Existing Knowledge → Candidate Discovery → Multi-Signal Analysis → Relationship Scoring → Evidence Validation → Persist Relationship

This approach makes the relationships more explainable and reduces false connections.

1. Candidate Relationship Discovery

The first step is determining which documents or knowledge objects are worth comparing.

Comparing every document with every other document would become expensive as the repository grows.

Instead, when a document is created or updated, the system can use its existing embedding to retrieve a small number of semantically close candidates.

For example:

New Document
↓
Vector search
↓
Top 20–30 possible related documents
↓
Detailed relationship analysis

This turns an expensive all-to-all comparison into a manageable candidate-ranking problem.

The current semantic retrieval infrastructure can therefore be reused rather than building another search mechanism.

2. Multi-Signal Relationship Analysis

Once candidate documents are found, the system should examine several independent signals.

Semantic similarity

Embedding similarity remains useful because two documents may discuss the same concept using completely different wording.

But semantic similarity should be treated as **one signal**, not the final answer.

Topic overlap

If two documents have been classified under the same or closely related topics, that increases the probability of a meaningful relationship.

Existing automatic classification and topic associations can be reused here.

Entity overlap

Suppose two documents independently reference:

Project Apollo
Microsoft Azure
Authentication Service
John Smith

Even if their text is different, the shared entities provide strong evidence that the documents belong to the same organizational context.

Entity overlap can therefore strengthen a relationship discovered through embeddings.

Project context

Documents connected to the same project are likely to have stronger organizational relevance than two semantically similar documents from unrelated projects.

Project relationships should therefore influence the relationship score.

Decisions and lessons

This is where the feature becomes particularly valuable.

A design document might describe an implementation.

A later incident report might explain why that implementation failed.

A lessons-learned record might explain how the organization changed its approach afterward.

Those three knowledge objects are much more useful when the KMS understands them as a connected chain rather than three independent search results.

3. Relationship Types

The system should avoid storing everything simply as RELATED_TO.

Where enough evidence exists, relationships can have meaningful types such as:

SIMILAR_TO, **SHARES_TOPIC**, **SHARES_ENTITY**, **RELATED_TO_PROJECT**, **SUPPORTS_DECISION**, **CONTRADICTS**, **SUPERSEDES**, or **RELATED_TO_LESSON**.

The exact initial set should remain small so the system does not become unnecessarily complex.

A generic relationship can still be used when the evidence shows a connection but cannot reliably classify its exact meaning.

4. Relationship Confidence

Each inferred relationship should have a confidence score.

Conceptually:

Relationship confidence = semantic similarity + topic evidence + entity evidence + project context + structured knowledge evidence

The weights should be configurable rather than permanently hard-coded.

For example:

Security Architecture → OAuth Migration Plan
Confidence: **0.91**
Evidence: same project, authentication topic, three shared entities, high semantic similarity.

This is substantially more useful than simply saying:

“These documents are related.”

5. Explainability and Provenance

For an enterprise system, this is one of the most important parts of the implementation.

The KMS should be able to explain **why** it believes a relationship exists.

For example:

Why are these related?
Same project: Identity Modernization
Shared topic: Authentication
Shared entities: OAuth, Azure AD
Semantic similarity: High

This also helps users distinguish between explicit relationships already known by the system and relationships inferred automatically by Rank 12.

The existing relationship evidence field can potentially support this without introducing unnecessary schema changes.

6. Background Processing

Relationship discovery should not increase normal upload or chat latency.

The preferred flow is:

Document created/updated
→ Existing ingestion completes
→ Relationship-discovery job triggered
→ Candidate relationships retrieved
→ Relationships scored
→ Strong relationships persisted

The user should not have to wait for all relationship calculations before the document becomes available.

Relationships can also be periodically recalculated when topics, projects, or other organizational context changes.

7. API and Codebase Integration

Rather than putting relationship logic directly into API routes, I recommend creating a dedicated service layer, conceptually:

src/lib/relationshipDiscovery.js

Its responsibilities would include candidate retrieval, signal calculation, scoring, evidence generation, thresholding and persistence.

API routes would then remain relatively thin.

Possible endpoints include:

GET /api/documents/[id]/relationships

for document-level relationships, and:

GET /api/org/[orgId]/relationships

for broader relationship discovery.

This follows the existing KMS architecture better than creating a separate service or microservice solely for Rank 12.

8. User Experience

A full visual graph is **not necessary for Rank 12** because that overlaps with Rank 13.

Instead, the document page could introduce a **Related Knowledge** or **Hidden Connections** panel.

For example:

Related Knowledge

Security Architecture
92% confidence
Same project · Authentication · 4 shared entities

OAuth Migration Decision
87% confidence
Related decision · Same topic

Authentication Incident Lesson
81% confidence
Related lesson · Shared project

Users should be able to select a result and navigate directly to the source.

9. Security Consideration

Relationship discovery must respect the existing KMS access-control model.

A relationship itself must never become a way to discover restricted information.

If a user can access Document A but cannot access Document B, the API should not expose Document B merely because the system discovered a relationship between them.

Therefore:

Relationship visibility ≤ visibility of the underlying knowledge.

Existing organization, department and document authorization logic should be reused rather than creating separate relationship permissions.

10. Success Criteria

I would consider the first version successful if it can:

1. automatically discover relationships for newly processed documents;
2. use multiple signals instead of embedding similarity alone;
3. assign confidence to inferred relationships;
4. preserve evidence explaining each relationship;
5. avoid creating excessive low-quality relationships;
6. respect existing RBAC;
7. run without noticeably affecting upload/chat latency;
8. expose useful related knowledge to users.

Final Recommendation

Rank 12 should be implemented as an **intelligence layer over the current KMS**, not as another retrieval engine.

The core design principle should be:

Retrieve candidates cheaply, validate relationships using multiple organizational signals, store only meaningful connections, and preserve the evidence behind them.

That creates the foundation required for Rank 13 and future graph-based reasoning.